

3DGRT

1 目的

3DGS 一般使用光栅化的方法对高斯核进行渲染，本文采用光线追踪的方法对高斯核进行渲染。光线追踪不仅便于实现阴影、反射、景深、相机扭曲、卷帘快门等效果，而且论文证明了能达到实时渲染的帧率。

2 前置知识

2.1 3D 高斯参数化

3D 高斯核对应的核函数为：

$$\rho(\mathbf{x}) = e^{-(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

其中 $\boldsymbol{\mu} \in \mathbb{R}^3$ 为高斯核的位置， $\boldsymbol{\Sigma} \in \mathbb{R}^{3 \times 3}$ 为高斯核的协方差矩阵。为了保持梯度下降时协方差矩阵的半正定性，将协方差矩阵进行如下表示：

$$\boldsymbol{\Sigma} = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T$$

其中 $\mathbf{R} \in SO(3)$ 为旋转矩阵， $\mathbf{S} \in \mathbb{R}^{3 \times 3}$ 为缩放矩阵。实际应用中，用四元数 $\mathbf{q} \in \mathbb{R}^4$ 表示旋转，用 $\mathbf{s} \in \mathbb{R}^3$ 表示缩放。

每个高斯核对应半透明系数 $\sigma \in \mathbb{R}$ 以及 radiance 函数 $\phi_{\boldsymbol{\beta}}(\mathbf{d}) : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ ， $\mathbf{d} \in \mathbb{R}^3$ 为观察方向。用三阶球谐函数 $Y_l^m(\mathbf{d})$ 表示，对应系数 $\boldsymbol{\beta} \in \mathbb{R}^3$ ：

$$\phi_{\boldsymbol{\beta}}(\mathbf{d}) = f\left(\sum_{l=0}^{l_{max}} \sum_{m=-l}^{m=l} \beta_l^m Y_l^m(\mathbf{d})\right)$$

f 为 sigmoid 函数，用于对 color 进行归一化。除此之外，此处的 $\mathbf{d}$ 使用的是光线方向，而 3DGS 原论文为了性能原因使用的是 $\frac{\boldsymbol{\mu} - \mathbf{o}}{\|\boldsymbol{\mu} - \mathbf{o}\|}$ 。

2.2 可微渲染

对于光线 $\mathbf{r}(\tau) = \mathbf{o} + \tau\mathbf{d}$ ，由经典体渲染公式：

$$\mathbf{L}(\mathbf{o}, \mathbf{d}) = \int_{\tau_n}^{\tau_f} T(\mathbf{o}, \mathbf{d}) \left(\sum_i (1 - e^{-\sigma_i \rho_i(\mathbf{o} + \tau\mathbf{d})}) \mathbf{c}_i(\mathbf{d}) \right) d\tau$$

其中 $\mathbf{c}_i(\mathbf{d}) = \phi_{\beta_i}(\mathbf{d})$ 对应第 i 个高斯核的 radiance 计算结果，transmittance 函数 $T(\mathbf{o}, \mathbf{d})$ 的定义为：

$$T(\mathbf{o}, \mathbf{d}) = e^{-\int_{\tau_n}^{\tau_f} \sum_i \sigma_i \rho_i(\mathbf{o} + t\mathbf{d}) dt}$$

考虑到 $\alpha_i = \sigma_i \rho_i(\mathbf{x}_i)$ ，体渲染公式可以用下式进行近似：

$$\mathbf{L}(\mathbf{o}, \mathbf{d}) = \sum_{i=1}^N \mathbf{c}_i(\mathbf{d}) \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

2.3 硬件光线追踪

本文使用 Optix 接口和 NVIDIA 显卡实现硬件光线追踪，支持光线和物体进行求交，包含以下 shader：

- ray-gen shader
- intersection shader
- any-hit shader
- closest-hit shader
- miss shader

3 算法核心

3DGRT 整个渲染流程如图1所示。

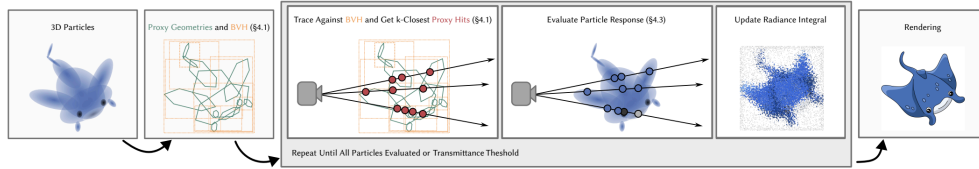


图 1: 3DGRT 的渲染流程

3.1 光线求交

Optix 支持三种图元添加到场景的 BVH 中，分别是三角形、球以及 AABB。此处可以用高斯核的包围球、包围盒或者能将高斯核包住的三角网格，但是需要考虑求交性能以及包围的紧密度。比如，如果高斯核不是轴对齐的，那么对应的 AABB 会有很多空余空间，导致求交判断会出现大量 miss，极大地降低求交效率。

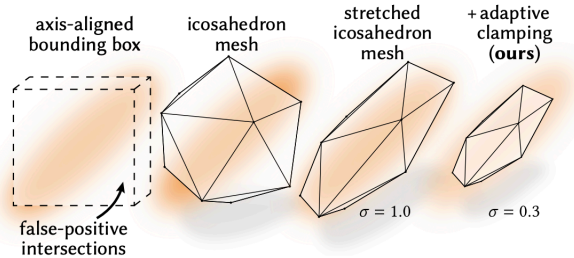


图 2: Geometry Proxy

论文中提到在进行和上述策略对比后，提出了 Stretched Polyhedron Proxy Geometry，通过正二十面体的网格对高斯核进行近似，如图2所示。设置最小相应 α_{min} (如 0.01)，构造二十个面的单位球，对每个顶点进行如下变换：

$$\mathbf{v} \leftarrow \sqrt{2\log(\sigma/\alpha_{min})} \mathbf{S} \mathbf{R}^T \mathbf{v} + \boldsymbol{\mu}$$

该放缩策略考虑了高斯核的 opacity，对于较大的但是接近透明的高斯核，会得到较小的包围图元，这里相当于对高斯核进行自适应地 clamp，降低采样开销。

3.2 光追渲染

- (1) 用 ray-gen 程序对场景的 BVH 进行遍历, 获取接下来的 k 个高斯核, 用 any-hit 程序维护一个根据深度排序的 buffer, 对应每个高斯核的索引;
- (2) 对这 k 个高斯核用体渲染方程进行渲染;
- (3) 从最后一个高斯核生成新光线, 继续收集 k 个高斯核;
- (4) 重复上述流程, 直到所有高斯核被处理完或者触发提前停止条件, 如达到最小 transmittance T_{min} 。

Procedure 1 RAY-GEN($\mathbf{o}, \mathbf{d}, T_{min}, \alpha_{min}, k, \tau_{SceneMin}, \tau_{SceneMax}$)

Input: ray origin $\mathbf{o}$, ray direction $\mathbf{d}$, min transmittance T_{min} , min particle opacity α_{min} , hit buffer size k , ray scene bounds $\tau_{SceneMin}$ and $\tau_{SceneMax}$

Output: ray incoming radiance L , ray transmittance T

```

1:  $L \leftarrow (0, 0, 0)$  ▷radiance
2:  $T \leftarrow 1.$  ▷transmittance
3:  $\tau_{curr} \leftarrow \tau_{SceneMin}$  ▷Minimum distance along the ray
4: while  $\tau_{curr} < \tau_{SceneMax}$  and  $T > T_{min}$  do
   ▷Cast a ray to the BVH for the next  $k$  hits, sorted
    $\mathcal{H} \leftarrow \text{TraceRay}(\mathbf{o} + \tau_{curr}\mathbf{d}, \mathbf{d}, k)$ 
6:
7: for  $(\tau_{hit}, i_{prim})$  in  $\mathcal{H}$  do ▷Render this batch of hits
8:    $i_{particle} \leftarrow \text{GetParticleIndex}(i_{prim})$ 
9:    $\alpha_{hit} \leftarrow \text{ComputeResponse}(\mathbf{o}, \mathbf{d}, i_{particle})$  ▷ $\sigma\rho(\mathbf{o} + \tau\mathbf{d})$ 
10:  if  $\alpha_{hit} > \alpha_{min}$  then
11:     $L_{hit} \leftarrow \text{ComputeRadiance}(\mathbf{o}, \mathbf{d}, i_{particle})$  ▷Refer to Equation 3 for SH evaluation
12:     $L \leftarrow L + T * \alpha_{hit} * L_{hit}$ 
13:     $T \leftarrow T * (1 - \alpha_{hit})$ 
14:     $\tau_{curr} \leftarrow \tau_{hit}$  ▷Resume tracing from last hit
15: end while
16: return  $L, T$ 

```

图 3: Ray Generation Program

Procedure 2 ANY-HIT($\tau_{hit}, i_{prim}, \mathcal{H}, k$)

Input: hit location τ_{hit} , primitive index i_{prim} , hit array $\mathcal{H}$, hit buffer size k

Output: the hit array $\mathcal{H}$ may be updated in-place with a new entry

```

1:  $h \leftarrow (\tau_{hit}, i_{prim})$ 
2: for  $i$  in  $0 \dots k-1$  do ▷insertion sort into hit array
3:   if  $h.\tau_{hit} < \mathcal{H}[i].\tau_{hit}$  then
4:      $\text{swap}(\mathcal{H}[i], h)$ 
5: ▷ignore  $k$ -closest hits to prevent the traversal from stopping
6: if  $\tau_{hit} < \mathcal{H}[k-1].\tau_{hit}$  then
7:    $\text{IgnoreHit}()$ 

```

图 4: Any Hit Program

3.3 计算高斯核的响应

论文对每个高斯核采样一个 sample，并计算该 sample 的响应。Knoll 曾提出用高斯核中心在光线上的投影，作为 sample 来计算响应，如图5所示。这种方法对各向同性的高斯核效果较好，但对各向异性的高斯核会带来较大误差。论文中提出计算 $\tau_{max} = \operatorname{argmax}_{\tau} \rho(\mathbf{o} + \tau \mathbf{d})$ ，获取高斯核在光线上的最大响应。代入高斯核方程，可以得到：

$$\tau_{max} = \frac{(\boldsymbol{\mu} - \mathbf{o})^T \boldsymbol{\Sigma}^{-1} \mathbf{d}}{\mathbf{d}^T \boldsymbol{\Sigma}^{-1} \mathbf{d}} = \frac{-\mathbf{o}_g^T \mathbf{d}_g}{\mathbf{d}_g^T \mathbf{d}_g}$$

其中 $\mathbf{o}_g = \mathbf{S}^{-1} \mathbf{R}^T (\mathbf{o} - \boldsymbol{\mu})$ ， $\mathbf{d}_g = \mathbf{S}^{-1} \mathbf{R}^T \mathbf{d}$ 。

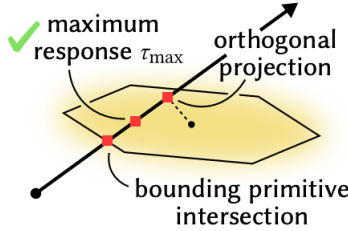


图 5: 高斯核响应

3.4 可微光追及优化

3.4.1 可微渲染

3DGRT 同样支持可微渲染，在前向阶段进行正常渲染并计算目标函数；在反向阶段，用相同的光线采样相同的高斯核，根据着色方程计算梯度，并用原子操作在共享的 buffer 中进行梯度累积。

3.4.2 优化

3DGRT 同样采用 3DGS 的 pruning、cloning 和 splitting 操作，但是 3DGS 是在屏幕空间进行梯度计算，而 3DGRT 使用三维世界空间进行梯度计算。

3.5 核函数

3DGRT 不强制使用高斯核函数，作者在论文中还测试了不同的核函数：

- 标准高斯核函数

$$\hat{\rho}(\mathbf{x}) = \sigma e^{-(\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})}$$

- 自由度为 n 的一般高斯核函数（论文中 $n = 2$ ）

$$\hat{\rho}_n(\mathbf{x}) = \sigma e^{-((\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}))^n}$$

- SGG2 曲面
- 沿着轴 i 建模的 $\cos$ 波

$$\hat{\rho}_c(\mathbf{x}) = \hat{\rho}(\mathbf{x})(0.5 + 0.5 \cos(\psi(\mathbf{S}^{-1}\mathbf{R}^T(\mathbf{x} - \boldsymbol{\mu})_i)))$$

其中 ψ 是优化参数。

4 参考

3DGRT